

UNIT II: JavaScript and Dynamic UI Development

JavaScript and DOM Manipulation: JavaScript fundamentals, events, DOM selection and updates, form validation, interactive UI behaviour.

Asynchronous Data Handling: Fetch API for API integration, JSON handling, loading states, error handling in UI, introduction to breaking problems into smaller steps.

JAVASCRIPT AND DOM MANIPULATION

JavaScript Fundamentals

What is JavaScript?

JavaScript is a scripting language, meaning it is written to be read and run directly by an engine rather than compiled into a separate executable ahead of time. It is also a dynamic language: the type of a variable is not fixed when it is declared but is determined and checked while the program is actually running. Every modern web browser ships with a JavaScript engine built in — Chrome and Edge use an engine called V8, Firefox uses SpiderMonkey, and Safari uses JavaScriptCore — and it is this engine that parses, compiles and executes JavaScript code.

JavaScript is designed to interact with an already-loaded webpage: it can read what is currently on screen, change it, remove parts of it, or add entirely new content, all without asking the browser to reload the page. By default JavaScript is client-side, which means it executes on the user's own device — inside their browser — rather than on the web server that originally sent the page. (JavaScript can also run outside the browser, for example on a server using an environment called Node.js, but this unit is concerned only with browser JavaScript.) A very common beginner confusion is worth clearing up immediately: JavaScript and Java are two completely different programming languages that happen to share part of a name for historical marketing reasons in the 1990s; knowledge of one does not automatically transfer to the other.

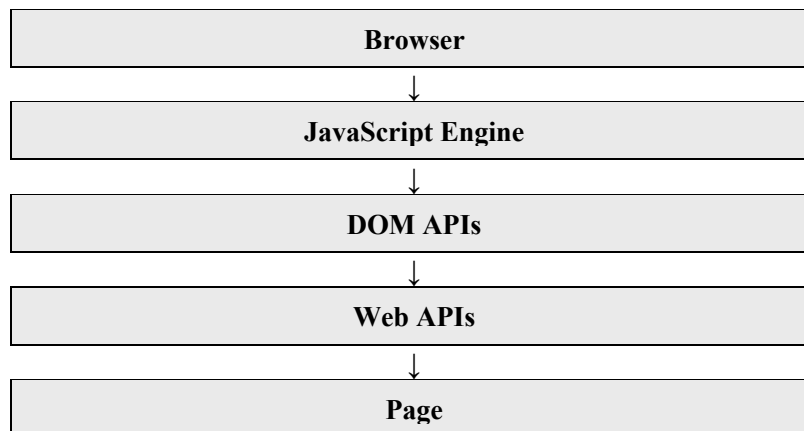
What Can JavaScript Do?

In a browser, JavaScript is responsible for essentially every behaviour a page has beyond its static appearance. Typical examples include: changing the text or styling of an element in response to an action; validating a form and showing an error message before the data is even submitted; responding to a click, a key press, or a mouse movement; creating brand-new elements and inserting them into the page, or removing existing ones; performing calculations, such as updating a shopping cart total; filtering a list of

data as a user types into a search box; communicating with a remote server to fetch fresh data; and updating the interface immediately, without the whole page reloading. Without JavaScript, a web page is essentially a static, printed poster; JavaScript is what allows it to behave like a genuine, responsive application.

JavaScript in the Browser — Language versus Browser APIs

It is important to separate two things that are both loosely called “JavaScript”. The first is the JavaScript language itself — variables, functions, loops, conditionals, objects — which is formally defined by a standard called ECMAScript and behaves identically wherever a JavaScript engine runs, whether inside a browser or elsewhere. The second is the collection of browser-provided APIs — tools such as `document`, `fetch`, `localStorage` and `setTimeout` — that are not part of the language definition at all, but are supplied by the browser as extra capabilities a script running inside it is allowed to use.



A simple test to tell the two apart: if a piece of functionality would still work inside a plain JavaScript program that has no browser at all (for example, running through Node.js on a server), then it belongs to the language. If it only works because a browser handed it to the running script, it is a browser API.

How JavaScript Connects HTML, CSS and the DOM

HTML, CSS and JavaScript are three separate technologies that cooperate to produce the final experience a user sees. HTML creates the raw elements on the page; CSS decides how those elements look; and JavaScript decides what happens when the user interacts with them. Consider a single button:

```
<!-- HTML: creates the button -->
<button>Save</button>
```

```
/* CSS: makes it look like a button */
```

```
button {  
  background: blue;  
}
```

```
// JavaScript: makes it DO something on click  
button.addEventListener("click", save);
```

HTML alone gives a button that exists but does nothing. CSS alone can make it look attractive but still does nothing when pressed. Only JavaScript can listen for the click and run a function such as `save()` in response. This division of responsibility — structure, presentation, behaviour — is one of the most important ideas in frontend development.

Three Ways to Add JavaScript to a Page

JavaScript can be attached to an HTML page in three different ways, each with its own trade-offs.

Method	Example	Notes
Inline	<code><button onclick="save()"></code>	Quick to try, but mixes HTML and logic together; avoided in real projects.
Internal	<code><script> ...code... </script></code>	No separate file needed, but the code cannot easily be reused on another page.
External	<code><script src="app.js"></script></code>	Reusable across pages, cacheable by the browser, and keeps HTML clean. This is the approach used in real projects.

Practical Tip: Using the Browser Console for Debugging

Every modern browser includes a set of built-in developer tools, opened with the F12 key or by right-clicking a page and choosing “Inspect”. The Console panel inside these tools is one of the most important tools available while learning JavaScript. The `console.log()` function prints a value to this panel, and liberally placing `console.log()` statements throughout a piece of code while it is being written is the single most common way beginners — and, in truth, experienced developers as well — check that a variable holds the value they expect at a particular point in the program.

```
console.log("score is:", score);  
console.log("user object:", user);
```

The console also displays any errors thrown while the page runs, along with the exact file name and line number where the error occurred, making it the first place to look whenever a script appears not to be working. The Elements or Inspector panel, covered in more detail once the DOM is introduced later in this unit, additionally allows the live DOM tree of the current page to be explored and even edited directly, which is invaluable for understanding exactly what a piece of JavaScript has changed.

External JavaScript and the defer Attribute

```
<script src="app.js" defer></script>
```

Without defer, the browser stops reading (parsing) the HTML the moment it reaches a `<script>` tag, downloads the script, runs it immediately, and only then continues reading the rest of the page. If that script tries to select an element that appears further down in the HTML — an element the browser has not read yet — the script will fail to find it. Adding the defer attribute tells the browser to keep downloading the script in the background while it continues parsing the HTML, but to wait until the entire HTML document has been parsed before actually running the script. By the time a deferred script runs, every element in the HTML is guaranteed to already exist. The related `async` attribute also downloads in the background, but runs the script the instant it finishes downloading — which may be before HTML parsing is complete, and the relative order of multiple `async` scripts is not guaranteed. For an ordinary page script, defer is therefore the safer and more predictable default.

Variables and Data Types

Variables: Storing Information

A variable is simply a named container that holds a value in the computer's memory while a program runs. Every application needs to remember things: a student's name, their marks, their age, a running total, whether they are currently logged in. Each of these is naturally stored in its own variable.

```
let studentName = "Asha";  
let marks = 87;  
let loginStatus = true;
```

var, let and const

Modern JavaScript provides three keywords for declaring a variable, and they differ in scope and in whether the variable can later be changed.

Feature	var	let	const
Scope	Function	Block	Block
Can be reassigned?	Yes	Yes	No
Can be redeclared?	Yes	No	No

Modern recommendation: start every variable as const; switch it to let only if the value genuinely needs to change later; avoid var altogether in new code, since its looser, function-level scoping is a common source of bugs. Note that const does not freeze the contents of an object or array — it only prevents the variable name itself from being pointed at a completely new value.

Scope in JavaScript

Scope determines where in a program a variable is visible and usable. Global scope refers to a variable declared outside any function or block; it is visible everywhere in the file. Function scope applies to variables declared with var inside a function; they are visible anywhere inside that function, but nowhere outside it. Block scope applies to variables declared with let or const inside a pair of curly braces — such as inside an if statement or a loop — and such variables are visible only within that specific block. Keeping variables in the smallest scope that is genuinely needed reduces the chance of two unrelated parts of a large program accidentally overwriting the same name, and makes bugs far easier to trace.

JavaScript Data Types

JavaScript defines seven primitive data types, each of which stores a single, simple value: String (text, such as "Asha"), Number (both whole numbers and decimals, such as 21 or 3.14), Boolean (only true or false), undefined (a variable that has been created but never assigned a value), null (a value deliberately set to represent “nothing here”), Symbol (a guaranteed unique value, used in advanced code) and BigInt (for integers too large for the ordinary Number type to represent precisely, written with a trailing n, such as 1234567890123456789n). In everyday form-driven web development, String, Number and Boolean are used constantly; the remaining types are less common but should still be recognised.

```
const name = "Asha";
const age = 21;
const passed = true;
```

A detail that regularly trips up beginners: the value typed into any text-based form field is always a String, even if the user typed digits. If a user types 25 into a number input, JavaScript initially sees the text "25",

not the number 25, and this must be explicitly converted (for example with the `Number()` function) before it can be used in arithmetic. A checkbox's `checked` property, by contrast, genuinely is a Boolean — `true` if ticked, `false` otherwise.

undefined versus null

	undefined	null
Meaning	A value has not been assigned yet	The absence of a value, set on purpose
Who sets it	JavaScript, automatically	The programmer, deliberately
Example	<code>let city; console.log(city); // undefined</code>	<code>let selectedUser = null; // no user chosen yet</code>

A simple way to remember the distinction: `undefined` usually means JavaScript did it by accident (a variable was declared but nobody ever filled it in), whereas `null` means the programmer did it on purpose (explicitly choosing to represent “empty”).

Template Literals

Template literals provide a cleaner way of building strings that mix fixed text with variable values. They are written using backtick characters (```) instead of ordinary quotation marks, and any expression placed inside `${ }` is automatically inserted into the string.

```
const name = "Asha";
const age = 21;
const message = `Hello ${name}, you are ${age} years old.`;
```

Compared with the older approach of joining strings using the `+` operator, template literals are shorter, easier to read, and support multi-line strings without any special escape characters. They are the standard way to build dynamic, data-driven text in a modern user interface.

Operators

Arithmetic Operators

JavaScript provides the usual set of arithmetic operators: `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `%` (remainder, also called the modulus operator) and `**` (exponentiation, i.e. “to the power of”).

```
const price = 500;
const qty = 3;
const total = price * qty;    // total = 1500

const change = 1500 % 400;    // change = 300
```

The % operator, the least familiar to most beginners, returns whatever is left over after one number is divided by another as many whole times as possible. 1500 divided by 400 fits three whole times (1200), leaving 300 remaining — that remainder is the answer. This operator is frequently used for tasks such as alternating the colour of every other row in a table, or working out pagination.

Assignment and Increment Operators

Operator	Meaning	Example (starting from x = 5)	Result
=	assign	x = 5	x is 5
+=	add and assign	x += 3	x is 8
-=	subtract and assign	x -= 2	x is 6
*=	multiply and assign	x *= 2	x is 12
/=	divide and assign	x /= 4	x is 3
++ / --	increment / decrement by 1	x++	x is 4

Each shorthand operator, such as +=, is simply a more concise way of writing x = x + 3. The ++ operator is extremely common inside loops, where it is used to advance a counter by one on every pass.

Comparison Operators and Type Coercion

JavaScript provides ==, ===, !=, !==, >, <, >= and <= for comparing values. The distinction between == and === is one of the most important rules in the entire language.

```
5 == "5"    // true
5 === "5"   // false
```

The == (loose equality) operator converts one value's type to match the other before comparing them, so it treats the number 5 and the string "5" as equal. The === (strict equality) operator compares both the value and the type, and correctly reports that a number and a string are never equal even if they look similar. This automatic type conversion performed by == is called type coercion, and because it happens

silently it is a common source of hidden bugs. The near-universal rule followed by professional JavaScript developers, and expected in this course, is: always use `===` and never use `==`.

Logical Operators

Operator	Meaning
<code>&&</code>	AND — true only if both sides are true
<code> </code>	OR — true if either side is true
<code>!</code>	NOT — flips true to false, or false to true

```
if (username && password) {  
  login();  
}  
// runs only if BOTH values exist
```

- `&&` is used to guard code that should only run when several conditions are simultaneously true — for example, only attempting a login when both a username and a password have been entered.
- `||` is used to supply a fallback value — for example, `const displayName = name || "Guest"` shows “Guest” if name happens to be empty.
- `!` is used to invert a boolean flag, such as toggling a menu between open and closed: `isOpen = !isOpen`.

Nullish Coalescing (??) and Optional Chaining (?.)

```
// Nullish coalescing  
const quantity = value ?? 1;  
// use value, unless it is null or undefined  
  
// Optional chaining  
user?.address?.city  
// returns undefined instead of crashing  
// if address does not exist
```

The `??` operator only substitutes its right-hand default when the left-hand value is specifically null or undefined; unlike `||`, it correctly leaves genuinely meaningful values such as 0 or an empty string untouched. The `?.` operator lets code read a deeply nested property safely: if any link in the chain (such as address) does not exist, the entire expression short-circuits to undefined instead of throwing an error and stopping the whole program. Both are modern, widely-supported features that make everyday code noticeably safer and shorter.

Truthy and Falsy Values

Inside a condition, JavaScript treats every value as either truthy or falsy. There are exactly six falsy values in the entire language:

false	0	"" (empty string)	null	undefined	NaN

Every other value — including the string "0", and an empty array [] — is truthy. This rule connects directly to form validation:

```
if (!name.value) {  
  showError("Name required");  
}  
// an empty string is falsy, so !name.value is true
```

Control Flow

Why Control Flow Is Needed

Real programs are rarely a straight sequence of instructions from top to bottom. They need to make decisions (choosing between two paths depending on a condition) and repeat work (running the same steps multiple times). Control flow structures — conditionals and loops — are what allow a script to behave like genuine logic rather than a fixed script that always does the same thing.

A	condition?	B or C
---	------------	--------

if / else

```
if (score >= 60) {  
  console.log("Pass");  
} else {  
  console.log("Fail");  
}
```

The block inside if runs only when its condition evaluates to true; the block inside else runs only when that same condition is false. The curly braces { } mark exactly which lines belong to each branch.

else if and Evaluation Order

```
if (marks >= 90) grade = 'A';  
else if (marks >= 75) grade = 'B';  
else if (marks >= 60) grade = 'C';  
else grade = 'F';
```

JavaScript checks each condition strictly from top to bottom, and the moment one condition is found to be true, it runs that branch and skips every remaining condition below it — even if those later conditions would also technically be true. Order therefore matters a great deal: if the check `marks >= 60` were placed before `marks >= 90`, a student who scored 95 would be incorrectly classified as grade C, because the 60-mark condition would be found true first and the program would never reach the more specific 90-mark check. The general rule is to place the strictest or most specific condition first, working down towards the least specific.

The Ternary Operator

```
const result = score >= 60 ? "Pass" : "Fail";
```

This single line is functionally identical to the five-line if/else example above, following the pattern `condition ? valueIfTrue : valueIfFalse`. Ternary expressions are best reserved for choosing a single, simple value; once a branch needs more than one line of logic, or ternaries begin to be nested inside one another, ordinary if/else is far more readable.

The switch Statement

```
switch (day) {  
  case 1:  
    console.log("Monday");  
    break;  
  default:  
    console.log("Unknown");  
}
```

- **case** checks a value for an exact match.
- **break** stops the switch from continuing into the next case block; omitting it is a very common source of bugs.
- **default** runs when none of the listed cases match, functioning like a final else.

switch is best suited to comparing a single value against several fixed, known options (such as a day number or a menu choice); if/else if remains the better tool for ranges or more complex conditions.

The for Loop

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

A for loop's parentheses contain three parts separated by semicolons: an initialisation that runs once at the very start (here, `let i = 0`); a condition checked before every single pass, which stops the loop the moment it becomes false (`i < 5`); and an update that runs after every pass (`i++`). Tracing this example by hand: the loop prints 0, 1, 2, 3 and 4 — five values — before `i` reaches 5, at which point the condition becomes false and the loop stops. A for loop is the natural choice whenever the number of repetitions is already known in advance, such as looping through every item in a fixed-size array.

The while Loop

```
let attempts = 0;  
while (attempts < 3) {  
  tryLogin();  
  attempts++;  
}
```

A while loop repeats its body for as long as its condition remains true, checking that condition before every pass. It is the appropriate tool when the number of repetitions is not known ahead of time and depends on something that may change unpredictably while the program runs — for example, the number of login attempts required before a user finally enters the correct password.

for...of and for...in

JavaScript provides two loop forms that iterate automatically over a collection, without needing a manual counter.

```
// for...of – iterates over VALUES (used with arrays)  
const fruits = ["Apple", "Mango"];  
for (const f of fruits) {  
  console.log(f);  
}
```

```
// Apple, Mango
```

```
// for...in – iterates over KEYS (used with objects)
const user = { name: "Asha", age: 21 };
for (const key in user) {
  console.log(key);
}
// name, age
```

A simple way to remember the difference: `for...of` retrieves the values contained within an array, while `for...in` retrieves the property names (keys) that exist inside an object.

Functions

Why Functions Matter

A function is a reusable, named block of logic that accepts some input, performs some work, and can return an output. Writing logic once inside a function and calling it wherever it is needed — rather than repeating the same code in several places — keeps a program organised, easier to test, and far easier to fix when a bug is found, since the correction only needs to be made in one location.

Input	Function	Output
-------	----------	--------

Function Declaration

```
function greet(name) {
  return `Hello ${name}`;
}

greet("Asha");    // "Hello Asha"
```

Term	Meaning
Function name	<code>greet</code>
Parameter	<code>name</code> — a placeholder written when the function is defined
Argument	<code>"Asha"</code> — the actual value supplied when the function is called

Term	Meaning
Return	sends a value back out of the function using the return keyword
Call	greet("Asha") — the moment the function actually runs

Function Expressions and Arrow Functions

```
// Function expression
const greet = function(name) {
  return `Hello ${name}`;
};

// Arrow function – a shorter equivalent
const greet = name => `Hello ${name}`;
```

A function expression stores an unnamed function inside a variable, rather than giving the function its own name as a declaration does; unlike a declaration, it cannot be called before it appears in the code (it is “not hoisted”). Arrow functions provide an even more concise syntax, and if the function body is a single expression with no curly braces, that value is returned automatically without needing the word return — this is called an implicit return. Arrow functions are extremely common in modern JavaScript, particularly as short callbacks passed into array methods and event listeners.

Default Parameters, Rest Parameters and Spread Syntax

```
// Default parameter
function multiply(a, b = 2) {
  return a * b;
}
multiply(5);      // 10
multiply(5, 3);   // 15
```

```
// Rest parameter – collects many arguments into one array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4);  // 10
```

```
// Spread syntax – expands an array/object into individual values
```

```
const nums = [1, 2, 3];  
Math.max(...nums);           // 3  
  
const base = { name: "Asha" };  
const user = { ...base, age: 21 };
```

Rest versus spread: both use the same three dots (...), but rest collects many separate values into a single array (used when defining a function), while spread expands one array or object out into many individual values (used when calling a function or building a new array/object).

Arrays

Array Basics

An array is an ordered collection of values stored under a single variable name. Every value inside an array has a position, called an index, and JavaScript arrays are zero-indexed — counting begins at 0, not 1. The property `length` reports how many items the array currently holds.

```
const fruits = ["Apple", "Banana", "Mango"];  
fruits[0];      // "Apple" (the FIRST item)  
fruits.length;  // 3
```

Basic Array Methods

Method / Property	Effect	Example
<code>push()</code>	adds an item to the end	<code>["A","B"].push("C") → ["A","B","C"]</code>
<code>pop()</code>	removes the last item	<code>["A","B","C"].pop() → ["A","B"]</code>
<code>includes()</code>	checks whether a value exists	<code>["A","B"].includes("A") → true</code>
<code>length</code>	the number of items	<code>["A","B","C"].length → 3</code>

`map()`, `filter()` and `reduce()`

These three methods are the workhorses of everyday array processing in JavaScript.

`map()` — transform every item into a new array

```
const prices = [10, 20, 30];  
const doubled = prices.map(p => p * 2);  
// [20, 40, 60]
```

`map()` runs a function on every item and returns a brand-new array of exactly the same length; the original array is never modified. It is ideal for turning raw data into UI-ready values, such as formatting every price for display.

***filter()* — keep only the items that pass a test**

```
const high = prices.filter(p => p > 20);  
// [30]
```

`filter()` keeps only the items for which the supplied test function returns true; the resulting array may be shorter than, or equal to, the original. This is exactly how a product catalogue's search or category filter works: `products.filter(p => p.category === "Books")`.

***reduce()* — combine an array down to one value**

```
const total = prices.reduce((sum, p) => sum + p, 0);
```

`reduce()` walks through the array carrying forward a running value, called the accumulator (here, `sum`), which starts at the value supplied as the second argument (`0`) and is updated on every step: `0 → 10 → 30 → 60`. This is precisely how a shopping cart calculates its total price from a list of item prices.

Other Common Array Methods

Method	Purpose	Example
<code>find()</code>	returns the first item matching a test	<code>users.find(u => u.id === 5)</code>
<code>forEach()</code>	runs code for each item; returns nothing	<code>items.forEach(i => render(i))</code>
<code>some()</code>	true if at least one item passes a test	<code>cart.some(i => i.stock === 0)</code>
<code>every()</code>	true only if every item passes a test	<code>form.every(f => f.valid)</code>
<code>sort()</code>	reorders the array in place	<code>nums.sort((a, b) => a - b)</code>

Objects

Object Basics

Where an array is a list accessed by numeric position, an object is a collection of related data accessed by name. Objects are ideal for grouping everything that describes a single entity — such as one student's record — under a single variable.

```
const user = {  
  name: "Asha",  
  age: 21,  
  isStudent: true  
};
```

- **property** — the label, e.g. name, age.
- **value** — the data stored, e.g. "Asha", 21.
- **method** — a function stored as a property (see below).

Accessing, Adding, Updating and Removing Properties

```
user.name          // dot notation  
user["name"]       // bracket notation
```

Dot notation is shorter and used most of the time. Bracket notation is required when the property name is itself stored inside a variable, or when it contains spaces or special characters:

```
const key = "age";  
user[key];    // works – reads user.age  
user.key;     // wrong – looks for a property literally named "key"
```

```
user.email = "a@example.com"; // adds a new property  
user.age = 22;                 // updates an existing one  
delete user.isStudent;         // removes the property entirely
```

Object Methods and this

```
const user = {  
  name: "Asha",  
  greet() {  
    return `Hi ${this.name}`;  
  }  
};  
user.greet(); // "Hi Asha"
```

A method is simply a function stored as a property inside an object. Inside a method, the keyword `this` refers to “the object the method was called on”, which allows a single method definition to work correctly

regardless of which object calls it. At this stage it is sufficient to understand `this.name` as meaning “this particular object's own name property”.

Destructuring

```
// Object destructuring
const { name, age } = user;

// Array destructuring
const [lat, lng] = coordinates;
```

Destructuring pulls values straight out of an object or array into their own individual variables in a single line, avoiding repeated `user.name`, `user.age` style access. It is especially common when writing function parameters, and when reading data returned from a web API, a topic covered later in this unit.

Worked Example: A Student Marks Calculator

The following short program brings together variables, arrays, functions, the `reduce()` array method and control flow into a single, realistic example — calculating a student's average marks across several subjects and deciding whether they have passed.

```
const marks = [78, 65, 92, 54, 88];

function calculateAverage(marksArray) {
  const total = marksArray.reduce((sum, m) => sum + m, 0);
  return total / marksArray.length;
}

function getResult(average) {
  return average >= 40 ? "Pass" : "Fail";
}

const average = calculateAverage(marks);
const result = getResult(average);

console.log(`Average: ${average.toFixed(2)}`);
console.log(`Result: ${result}`);
```

Reading this program from the top: `marks` is an array holding five subject scores. `calculateAverage()` uses `reduce()` to add every mark together and then divides by the number of subjects to produce the average — exactly the pattern introduced earlier in this section. `getResult()` applies a simple ternary condition to

decide the outcome. Finally, template literals are used to print a friendly, readable summary, with the built-in `toFixed(2)` method rounding the average to two decimal places. This short example illustrates a general truth about JavaScript: complex-looking behaviour is almost always built from a small number of simple, well-understood building blocks combined together, rather than from any single complicated feature.

Common Mistakes to Avoid — JavaScript Fundamentals

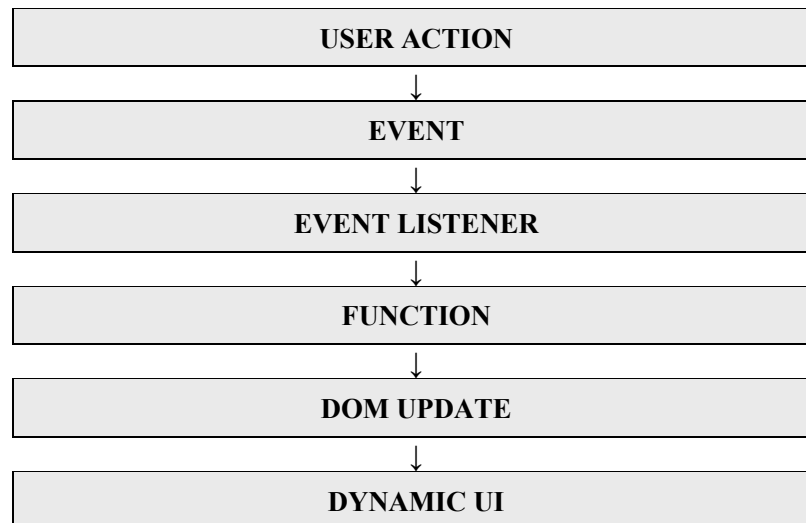
Before moving on to events and the DOM, it is worth summarising the mistakes that beginners make most often with the material covered so far, since recognising them in advance prevents a great deal of confusion later.

- **Using `==` instead of `===`:** always prefer strict equality, since loose equality silently converts types and can compare values that appear unrelated as if they were equal.
- **Forgetting that `.value` is always a string:** a number typed into a form field must be explicitly converted with `Number()` before it can be used in arithmetic.
- **Confusing undefined and null:** undefined normally means a value was never set; null means a value was deliberately set to nothing.
- **Using `var` in new code:** its function-level scope is looser than the block-level scope of `let` and `const`, and it can quietly cause bugs that are difficult to trace.
- **Assuming `const` freezes an object:** `const` only stops the variable name from being reassigned; the properties of an object or the items of an array it points to can still be changed freely.
- **Placing conditions in the wrong order:** in a chain of `if/else if` statements, the first true condition wins and all later ones are skipped, so the most specific condition should always be checked first.
- **Forgetting `break` in a `switch` statement:** without it, execution falls through and continues running the code belonging to the next case as well.
- **Mutating an array while expecting the original to stay the same:** methods such as `map()` and `filter()` always return a new array and leave the original untouched, whereas methods such as `push()` and `sort()` change the original array directly (“in place”).

Events

The Events-to-UI Chain

Every piece of dynamic behaviour on a webpage follows the same underlying chain: a user performs an action, the browser raises an event describing what happened, a previously-registered event listener is triggered, that listener runs a JavaScript function, the function updates the DOM, and the user immediately sees the resulting change on screen.



The `addEventListener` method is used to attach a function to an element so that it runs whenever a particular kind of event occurs on that element:

```
button.addEventListener("click", () => {  
  title.textContent = "Clicked!";  
});
```

Some of the most frequently used event types are summarised below.

Event	Fires when...
click	the user presses and releases a mouse button (or taps) on an element
input	the value of a text field changes, on every keystroke
change	a field's value changes and then loses focus (common with select, checkbox, radio)
submit	a form is submitted
keydown / keyup	a keyboard key is pressed down / released
focus / blur	an element gains / loses keyboard focus
mouseenter / mouseleave	the mouse pointer enters / leaves an element

Event	Fires when...
DOMContentLoaded	the initial HTML has been fully parsed (fired on document)

The Event Object

```
button.addEventListener("click", (event) => {  
  console.log(event.type);    // "click"  
  console.log(event.target);  // the exact element clicked  
});
```

Every event listener automatically receives an event object as its argument, describing precisely what happened. `event.type` reports the kind of event that occurred, and `event.target` reports the exact element on which the event originated — which is especially useful when a single listener is attached to a parent element that contains many children, since `event.target` reveals which specific child was actually interacted with. It is worth noting that inside an arrow-function listener, the keyword `this` does not refer to the element the listener is attached to (arrow functions do not have their own `this`); `event.target` should be used instead whenever the triggering element is needed.

Event Bubbling and Capturing

When an event occurs on an element, it does not only fire on that element. It also fires again on that element's parent, then on the parent's parent, and so on all the way up to the document itself. This behaviour is called event bubbling, and it is the default for the great majority of events.

```
document  
  body  
    div.card  
      button  <-- clicked here  
  
Bubbling path: button -> div.card -> body -> document
```

In fact, every event technically travels through two phases: a capturing phase, which travels from the top of the document down towards the element that triggered the event, followed by the bubbling phase, which travels back up from that element to the top. By default, `addEventListener` listens during the bubbling phase; passing an options object of `{ capture: true }` as a third argument makes the listener fire during the capturing phase instead, though this is rarely required in ordinary application code.

stopPropagation() and preventDefault()

These two methods are frequently confused, but they control entirely different things.

```
button.addEventListener("click", (e) => {  
  e.stopPropagation();  
  // stops the event from bubbling up to parent listeners  
});
```

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  // stops the browser's own default behaviour  
  // (e.g. reloading the page on form submit)  
});
```

Do not confuse the two: stopPropagation() controls where an event is allowed to travel (it stops the event reaching parent listeners), while preventDefault() controls the browser's built-in default reaction to that event (such as a form auto-submitting and reloading the page, or a link navigating away). A form's submit handler almost always calls preventDefault() so that JavaScript, rather than a full page reload, is responsible for handling the submission.

Event Delegation

```
// ONE listener on the parent, instead of one per <li>  
list.addEventListener("click", (e) => {  
  if (e.target.tagName === "LI") {  
    e.target.classList.toggle("done");  
  }  
});
```

Event delegation deliberately makes use of bubbling: instead of attaching a separate listener to every individual child element (such as every item in a to-do list), a single listener is attached to the shared parent, and event.target is used to work out which child was actually clicked. This has two practical benefits. First, it automatically works for items that are added to the page later — there is no need to attach a fresh listener to every new to-do item as it is created. Second, it results in far fewer listeners overall, which matters for performance on long lists.

removeEventListener()

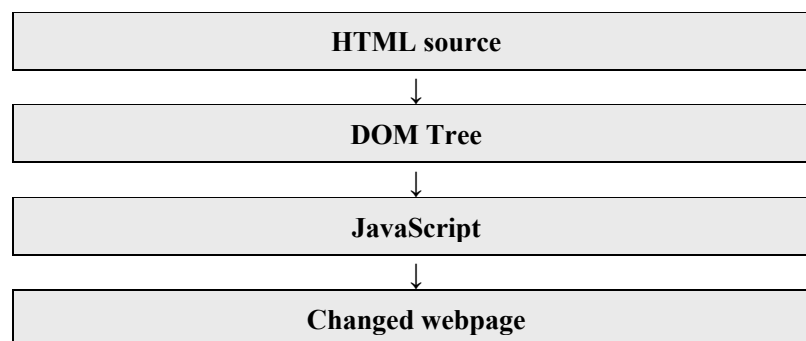
```
function handleClick() {  
  console.log("clicked");  
}  
  
button.addEventListener("click", handleClick);  
button.removeEventListener("click", handleClick);
```

`removeEventListener` requires the exact same function reference that was originally passed to `addEventListener`; it will not work if an anonymous arrow function was written directly inline, since there is no way to refer back to it afterwards.

DOM Selection and Updates

What is the DOM?

DOM stands for Document Object Model. When a browser loads an HTML document, it does not simply display the raw text — it builds a live, in-memory tree of objects that represents every element on the page. JavaScript never edits the original HTML file on disk; it edits this in-memory tree, and the browser instantly re-draws whatever is currently visible the moment the tree changes. This is what makes a page feel alive without ever needing to reload.



DOM Tree Structure

```
document  
  -- html  
    |-- head  
    +-- body  
      |-- h1  
      |-- p  
      +-- button
```

Every tag in an HTML document becomes a node in this tree, and the standard family vocabulary is used to describe the relationships between nodes: body is the parent of h1, p and button; h1 is a child of body; and h1, p and button, sharing the same parent, are siblings of one another.

Selecting Elements

Method	Returns
document.querySelector()	the first element matching a CSS-style selector
document.querySelectorAll()	all matching elements, as a NodeList
document.getElementById()	a single element with the given id
document.getElementsByClassName()	a live collection of elements with a given class

In modern JavaScript, querySelector and querySelectorAll are the most commonly used tools, since both accept the same selector syntax already familiar from CSS. Consider the following HTML:

```
<h1 id="title">Hello</h1>
<button class="btn">One</button>
<button class="btn">Two</button>
```

```
document.querySelector("#title");    // returns ONE element
document.querySelectorAll(".btn");   // returns a NodeList of TWO elements
```

Reading and Updating Content: textContent versus innerHTML

```
title.textContent = "Welcome Students";

box.innerHTML = "<strong>Hello</strong>";
```

textContent sets plain text only; any HTML tags placed inside the string are displayed as literal characters rather than being rendered. innerHTML, by contrast, genuinely parses and renders real HTML markup.

Security warning: raw, unsanitised text typed by a user should never be inserted using innerHTML, because a malicious user could type actual HTML or script content that would then be executed on the page — a vulnerability known as Cross-Site Scripting (XSS). textContent should always be used for displaying plain user-supplied text, since it can never be interpreted as executable markup.

Creating, Inserting and Removing Elements

Beyond reading and updating existing elements, JavaScript can also build entirely new elements from scratch and insert them into the page, or remove elements that are no longer needed. This is essential whenever a page needs to display a variable, data-driven list — such as search results, comments, or products — rather than a fixed number of pre-written HTML elements.

```
// Create a new element
const item = document.createElement("li");
item.textContent = "New To-Do Item";

// Insert it into the page
list.appendChild(item);

// Remove an element
item.remove();

// Older / alternative way to remove a child
list.removeChild(item);
```

Method	Purpose
document.createElement(tagName)	creates a brand-new element in memory, not yet visible on the page
parent.appendChild(child)	inserts an element as the last child of a parent, making it visible
element.remove()	removes the element itself from the page
parent.removeChild(child)	removes a specific child from a specific parent (older syntax)

A very common pattern combines creation with a loop: an empty container is selected, its existing contents are cleared (often by setting `innerHTML` to an empty string), and then a new child element is created and appended for every item in a data array. This exact pattern appears again later in this unit when API data is rendered onto the page.

Attributes and classList

```
setAttribute() / getAttribute() / removeAttribute()

classList.add()
classList.remove()
classList.toggle()
classList.contains()
```

setAttribute, getAttribute and removeAttribute allow any HTML attribute (such as src, href, disabled or placeholder) to be read, set or removed from JavaScript. The classList family of methods specifically manages an element's CSS classes: add() and remove() add or remove a single class; toggle() adds a class if it is not currently present and removes it if it is, which is ideal for on/off switches such as a dark-mode button; and contains() checks whether a class is currently applied, returning true or false.

Button clicked	JS runs	class added	CSS rule applies	UI changes
----------------	---------	-------------	------------------	------------

This sequence directly connects DOM manipulation back to the CSS covered in Unit I: JavaScript's role here is only to add or remove a class at the correct moment; the actual visual styling continues to be defined entirely by CSS.

Form Validation

Why Form Validation Matters

Users regularly make mistakes when filling in forms: typing errors, empty required fields, or values in the wrong format. Poor validation allows bad data to reach the server, where it can break the application or corrupt stored records. Validation exists in two places: the browser (client-side) and the server (server-side).

Important distinction: client-side JavaScript validation exists purely to give the user a good experience — instant, friendly feedback before they even submit the form. It is not security, because a determined user can bypass browser-based JavaScript entirely. The server must always re-check every value again before it is saved, regardless of what the browser already validated.

Accessing Form Values in JavaScript

```
<input id="email" type="text">

const emailField = document.querySelector("#email");
emailField.value; // gets the CURRENT text typed by the user, as a String
```

Every form control exposes its current value through the .value property, which is always a string — even for a number input, meaning Number() must be used before performing arithmetic on it.

Custom Validation Logic

```
function validateName(value) {  
  if (!value.trim()) {  
    return "Name is required";  
  }  
  return null; // no error  
}
```

A clean pattern is to write one small function per field that returns an error message string when the value is invalid, or null when it is valid. The `.trim()` method removes leading and trailing spaces, so a value consisting only of spaces is correctly treated as empty too. This keeps validation logic organised, reusable and easy to test.

Regular Expressions for Validation

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  
emailPattern.test("a@b.com"); // true  
emailPattern.test("not-an-email"); // false
```

A regular expression (regex) is a pattern used to match text, functioning like a highly precise “find” rule. The `.test()` method checks whether a given string matches the pattern and returns true or false. At this level, students are expected to be able to use an existing regular expression for a common task such as email validation, rather than write complex patterns from scratch.

Real-Time Validation with the input Event

```
emailField.addEventListener("input", () => {  
  const error = validateEmail(emailField.value);  
  showError(error);  
});
```

The input event fires on every single keystroke, giving instant feedback as the user types. The related change event, by contrast, fires only once the field loses focus. Real applications often combine both: live feedback via input, with a final, authoritative check repeated again on submit.

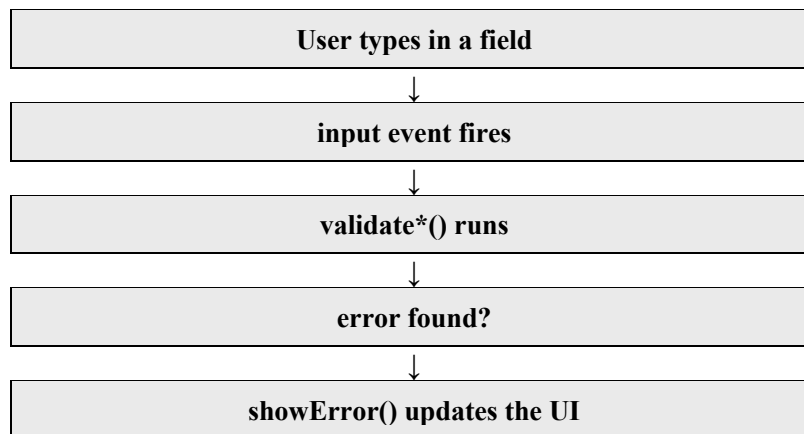
Showing and Hiding Error Messages

```
function showError(message) {
```

```
const box = document.querySelector("#emailError");
if (message) {
  box.textContent = message;
  box.classList.add("visible");
} else {
  box.classList.remove("visible");
}
}
```

A Full Login Form Validation Example

Bringing these pieces together, a typical login form is validated as follows.



```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  if (hasErrors()) return;
  submitForm();
});
```

Interactive UI Behaviour

Combining DOM, Events and Arrays

The techniques covered so far — selecting elements, listening for events, and manipulating arrays with methods such as `filter()` — combine naturally to build genuinely interactive interface behaviour. A live search box that filters a product list as the user types is a representative and very common example.

```
const searchBox = document.querySelector("#search");
const list = document.querySelector("#list");
```

```
searchBox.addEventListener("input", () => {
  const term = searchBox.value.toLowerCase();
  const results = allProducts.filter((p) =>
    p.name.toLowerCase().includes(term)
  );
  renderProducts(results);
});
```

Every keystroke in the search box triggers the input event, which reads the current text, filters the full product array down to only the matching items, and re-renders just those results. Other everyday examples of interactive UI behaviour built from exactly the same core ingredients — selection, events and DOM updates — include a to-do list where items can be marked complete or removed, a character counter beneath a text box, a collapsible FAQ section, and a tabbed interface that shows or hides different panels of content.

Worked Example: A To-Do List with Event Delegation

A to-do list is a classic beginner project that exercises almost every DOM and event technique covered in this unit at once: creating elements, appending them to the page, listening for clicks, and removing elements. Rather than attaching a fresh listener to every individual to-do item — which would fail for items added later — a single delegated listener is attached to the shared list container.

```
const input = document.querySelector("#newTask");
const addButton = document.querySelector("#addTask");
const list = document.querySelector("#taskList");

addButton.addEventListener("click", () => {
  if (!input.value.trim()) return;

  const item = document.createElement("li");
  item.textContent = input.value;

  const removeBtn = document.createElement("button");
  removeBtn.textContent = "Delete";
  item.appendChild(removeBtn);

  list.appendChild(item);
  input.value = "";
});

// ONE delegated listener handles clicks on
```

```
// EVERY current and future task and delete button
list.addEventListener("click", (e) => {
  if (e.target.tagName === "BUTTON") {
    e.target.closest("li").remove();
  } else if (e.target.tagName === "LI") {
    e.target.classList.toggle("done");
  }
});
```

The `addButton` listener creates a new list item and a delete button for it whenever the Add button is pressed, guarding against adding an empty task with the same falsy-value check introduced earlier for form validation. The single delegated listener on `list` then handles two different behaviours depending on exactly what was clicked: if a Delete button was clicked, the `closest()` method walks upward from that button to find its containing `li` element and removes it; if the `li` text itself was clicked, its `done` class is toggled, which could be paired with a CSS rule to draw a line through completed tasks. Because the delegated listener is attached to `list` itself rather than to any individual item, it automatically continues to work correctly for every new task added after the page first loaded.

Worked Example: A Live Character Counter

A character counter beneath a text box — commonly seen under bio or comment fields with a maximum length — is a small but instructive example of the input event combined with a simple calculation.

```
const bio = document.querySelector("#bio");
const counter = document.querySelector("#charCount");
const MAX = 150;

bio.addEventListener("input", () => {
  const remaining = MAX - bio.value.length;
  counter.textContent = `${remaining} characters remaining`;
  counter.classList.toggle("warning", remaining < 10);
});
```

On every keystroke, the current text's length is subtracted from the maximum allowed length to work out how many characters remain, and that value is written into the counter element using `textContent`. The second argument to `classList.toggle()` is a boolean: when it is true the class is added, and when false it is removed, which here is used to automatically highlight the counter once fewer than ten characters remain — without needing a separate `if/else` block.

A consistent thread runs through all of these examples: an event is detected, a small amount of JavaScript logic decides what should change, and the DOM is updated to reflect that decision. Mastery of interactive UI behaviour, at this stage of learning, is less about learning new syntax and more about fluently combining the DOM, events and array-processing techniques already covered.

Common Mistakes to Avoid — Events, DOM and Forms

- **Selecting an element before it exists:** a script placed in `<head>` without `defer`, or run before the relevant HTML has loaded, will select nothing and silently fail; placing scripts at the end of `<body>`, or using `defer`, avoids this.
- **Using `innerHTML` for plain user text:** always prefer `textContent` for untrusted or user-supplied text to avoid the risk of Cross-Site Scripting.
- **Attaching a listener to every list item individually:** this fails for items added later; event delegation on a shared parent solves this cleanly.
- **Forgetting `preventDefault()` on a form's submit event:** without it, the browser performs its own default action — typically reloading the page — before JavaScript has a chance to handle the data.
- **Relying only on placeholder text instead of a real `<label>`:** placeholder text disappears the moment the user starts typing and is not a substitute for a properly associated label, both for accessibility and for basic usability.
- **Validating only on submit and never while typing:** combining the input event for live feedback with a final check on submit produces a noticeably better user experience.

ASYNCHRONOUS DATA HANDLING

Introduction to Asynchronous JavaScript

Synchronous versus Asynchronous Code

Synchronous code runs strictly one line after another; each line waits for the previous one to finish before it begins. Asynchronous code allows a slow operation — such as a network request, a timer, or reading a large file — to run in the background while the rest of the program continues without waiting for it.

Line 1	Line 2	Line 3
--------	--------	--------

In synchronous code, each step above waits for the previous one. In asynchronous code, a slow step (such as fetching data) is started and the program moves on immediately, without freezing, and is only notified with the result once it is ready.

Why Asynchronous Code Is Necessary

JavaScript in the browser runs on a single thread, meaning it can only execute one instruction at a time. If a slow operation such as a network request were allowed to block that single thread until it finished, the entire page — including scrolling, clicking, and typing — would freeze completely for however long the operation took. A frozen page is a poor user experience, and this is precisely the problem that asynchronous JavaScript solves. Real slow operations that benefit from being asynchronous include fetching data from an API over the internet, reading a large file, waiting for a timer, and waiting for a database query to complete.

The Underlying Mechanism (Simplified)

Although JavaScript itself is single-threaded, the browser surrounding it provides additional systems that work together to keep the page responsive: Web APIs (browser-provided facilities that can run tasks in the background), a callback or task queue (which holds the results of finished background tasks), and an event loop (which continuously checks whether the main thread is free, and if so, moves the next waiting result from the queue onto it). For the purposes of this course, the simplified idea to remember is that a slow task is handed off to the browser, and the JavaScript code is notified with the result later, once it is ready, without ever freezing anything in the meantime.

JS Engine (Call Stack)	Web APIs (browser)	Callback Queue	Event Loop
------------------------	--------------------	----------------	------------

setTimeout — a Simple Asynchronous Example

```
console.log("1");
setTimeout(() => {
  console.log("2");
}, 1000);
console.log("3");

// Output order: "1", "3", "2" (after about 1 second)
```

setTimeout does not pause the program; it merely schedules its function to run later and immediately allows the rest of the code to continue. This is why "3" is printed before "2": line five does not wait for the one-second timer to finish.

Callback Functions

```
function loadData(callback) {
  setTimeout(() => {
    callback("Data loaded!");
  }, 1000);
}

loadData((result) => {
  console.log(result);
});
```

A callback is a function passed into another function so that it can be run later, once some asynchronous work has finished. This was the traditional way of handling asynchronous results in earlier JavaScript, and it remains a useful concept to understand even though newer tools now largely replace it.

The Callback Hell Problem

```
getUser(id, (user) => {
  getOrders(user, (orders) => {
    getInvoice(orders, (invoice) => {
      // deeply nested, hard to read,
      // and hard to handle errors
    });
  });
});
```

When several asynchronous steps must happen in sequence, nesting one callback inside another quickly produces code that grows sideways rather than downward, sometimes nicknamed the “pyramid of doom”. Error handling also becomes messy, since every nested level needs its own separate checks. This exact readability and error-handling problem is what motivated the introduction of Promises, covered next.

Promises

What is a Promise?

A Promise is an object that represents a value which will become available at some point in the future — the eventual outcome of an asynchronous operation. It can be thought of as a delivery-tracking receipt: the item itself is not in hand yet, but there is a firm promise that it will either arrive successfully or that a failure notice will be issued instead.

The Three Promise States

Pending	Fulfilled	Rejected
---------	-----------	----------

Every Promise begins in the pending state, meaning the operation is still underway. It then settles exactly once, permanently, into either fulfilled (the operation succeeded, and a resulting value is available) or rejected (the operation failed, and a reason or error is available). Once a Promise has settled, its outcome can never change again.

Creating a Promise

```
const promise = new Promise((resolve, reject) => {  
  const success = true;  
  if (success) {  
    resolve("Done!");  
  } else {  
    reject("Something went wrong");  
  }  
});
```

`resolve(value)` marks the Promise as fulfilled with the supplied value, while `reject(reason)` marks it as rejected with the supplied reason. In everyday application code, Promises are rarely constructed by hand in this way — built-in browser functions such as `fetch()` already return ready-made Promises — but understanding this underlying mechanism clarifies what is actually happening whenever a Promise is used.

.then() and .catch()

```
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  });
```

- **.then()** runs when the Promise becomes fulfilled, and receives the resolved value as its argument.
- **.catch()** runs when the Promise becomes rejected, and receives the error or reason as its argument.

Chaining Promises

```
getUser(id)
  .then((user) => getOrders(user))
  .then((orders) => getInvoice(orders))
  .then((invoice) => {
    console.log(invoice);
  })
  .catch((error) => {
    console.log(error);
  });
```

Compared with the nested callback example seen earlier, this reads top to bottom in a straight, flat sequence rather than nesting sideways, and a single `.catch()` at the end is able to handle an error occurring at any step in the entire chain.

Promise.all()

```
Promise.all([
  fetch(url1),
  fetch(url2),
  fetch(url3)
]).then((results) => {
  // all three have now finished
});
```

`Promise.all()` runs several Promises simultaneously rather than one after another. Its `.then()` only fires once every Promise in the list has finished; if even one of them rejects, the entire `Promise.all()` immediately

rejects. This is useful whenever a page needs several independent pieces of data before it can usefully render anything.

Promises versus Callbacks — Summary

Callbacks	Promises
Nest sideways (“pyramid of doom”)	Chain downward, remaining readable
Error handling repeated at every level	A single <code>.catch()</code> covers the whole chain
No built-in way to run many tasks together	<code>Promise.all()</code> runs many tasks at once

async / await

What is async/await?

`async/await` is newer syntax, built directly on top of Promises, that allows asynchronous code to be written so that it looks and reads like ordinary, synchronous, top-to-bottom code. It does not replace Promises — every function marked `async` still returns a Promise behind the scenes — it simply provides a cleaner way of writing and reading them. The rule worth memorising: `async/await` is just a nicer way of writing `.then()/.catch()` chains; both approaches do exactly the same thing underneath.

The async Keyword

```
async function loadData() {  
  return "Hello";  
}  
  
loadData().then((value) => {  
  console.log(value);    // "Hello"  
});
```

Placing `async` before a function definition automatically makes that function return a Promise. Even though the function body simply writes `return "Hello"`, calling `loadData()` actually produces a Promise that resolves to the string `"Hello"`.

The await Keyword

```
async function getUser() {  
  const response = await fetch("/api/user");  
  const data = await response.json();  
  console.log(data);  
}
```

```
}
```

`await` pauses execution of the surrounding function at that exact line until the Promise on its right-hand side settles — without freezing the rest of the page in the meantime. `await` can only be used inside a function that has itself been marked `async`. Everything written below an `await` line simply reads like ordinary, sequential code.

Error Handling with `try/catch`

```
async function getUser() {  
  try {  
    const response = await fetch("/api/user");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Failed:", error);  
  }  
}
```

`async/await` versus `.then()` — Side by Side

```
// .then() style  
fetch(url)  
  .then(r => r.json())  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

```
// async/await style  
try {  
  const r = await fetch(url);  
  const data = await r.json();  
  console.log(data);  
} catch (err) {  
  console.log(err);  
}
```

Both examples above perform exactly the same task. `async/await` is generally preferred in modern JavaScript for its readability, particularly once more than one asynchronous step must run one after another.

Fetch API for API Integration

What is the Fetch API?

The Fetch API is a built-in browser Web API used to request data from a server over the network, such as a REST API. It is the modern replacement for the older XMLHttpRequest technique, and it is the standard way a web application loads live data — weather information, product listings, user profiles, or comments.

Your Page	fetch()	Server / API	Response
-----------	---------	--------------	----------

Basic Fetch Syntax

```
fetch("https://api.example.com/products")
  .then((response) => response.json())
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.log("Error:", error);
  });
```

fetch() Returns a Promise

fetch(url)	Promise	Response object
------------	---------	-----------------

Calling fetch() immediately returns a Promise — not the data itself. That Promise resolves to a Response object as soon as the server has replied with its headers. The Response object's body still requires one further step to actually read, which is exactly the purpose of response.json().

Reading the Response — response.json()

```
const response = await fetch(url);
const data = await response.json();
// data is now a real JavaScript object/array you can use
```

The server transmits only raw text; response.json() reads that text and converts it into a genuinely usable JavaScript object or array. Notably, response.json() itself also returns a Promise, which is why it too is awaited separately from the original fetch() call. A related method, response.text(), is available for responses that are plain text rather than JSON.

A Full Fetch Example with async/await

```
async function loadProducts() {
  try {
    const response = await fetch("/api/products");
    const products = await response.json();
    renderProducts(products);
  } catch (error) {
    showError("Could not load products");
  }
}
```

HTTP Status Codes and response.ok

Code	Meaning
200	OK — success
201	Created — success (new resource)
400	Bad Request — client error
404	Not Found
500	Internal Server Error

`response.status` reports the numeric status code, and the boolean `response.ok` is true only for status codes in the 200–299 range.

Critical point (from MDN): `fetch()`'s Promise does not reject on an HTTP error response such as 404 or 500 — it only rejects on a genuine network failure (for example, no internet connection or a DNS lookup failure). A page that returns a 404 status still counts as a successfully completed fetch; `response.ok` or `response.status` must be checked manually to detect this kind of failure. Relying only on `.catch()` to detect a failed request is a common mistake, since it will not catch a 404 response.

Sending Data — POST Requests

```
fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Asha", age: 21
  })
});
```

Fetch Error Handling — Two Kinds of Failure

Network Error	HTTP Error
No internet, DNS failure, server unreachable	Server responded, but with an error status (404, 500)
Promise REJECTS — caught by .catch() / try-catch	Promise still RESOLVES — response.ok must be checked

```
if (!response.ok) {  
  throw new Error("Server error: " + response.status);  
}
```

Manually throwing an error when `response.ok` is false allows the same catch block to correctly handle both network errors and HTTP errors within a single, unified piece of error-handling code.

JSON Handling

What is JSON?

JSON stands for JavaScript Object Notation. It is a lightweight, text-based format for storing and exchanging data, and it is the standard format used by almost every web API to send and receive information. Although JSON text looks very similar to a JavaScript object or array, it remains pure text until it has been parsed.

```
{  
  "name": "Asha",  
  "age": 21,  
  "isStudent": true  
}
```

JSON Syntax Rules

- Property names must be written in double quotes — "name", not name or 'name'.
- String values must also use double quotes, never single quotes.
- No trailing comma is allowed after the final item in an object or array.
- JSON contains pure data only — it cannot contain functions or comments.
- Valid value types are: string, number, boolean, null, array and object.

JSON.parse() and JSON.stringify()

```
// text -> JS object
const json = '{"name":"Asha"}';
const obj = JSON.parse(json);
obj.name;    // "Asha"
```

```
// JS object -> text
const obj = { name: "Asha" };
const json = JSON.stringify(obj);
// '{"name":"Asha"}'
```

`JSON.parse()` converts JSON text into a real, usable JavaScript object, which is exactly what happens automatically inside `response.json()`. `JSON.stringify()` performs the reverse conversion, turning a JavaScript object into JSON text — used, for example, when sending data inside the body of a POST request.

Working with JSON Arrays of Objects

```
[
  { "id": 1, "name": "Pen", "price": 10 },
  { "id": 2, "name": "Book", "price": 150 }
]
```

This structure — an array of objects — is exactly the shape most real APIs return. Once such data has been parsed, all of the array techniques already covered can be applied directly:

```
products.map(p => p.name);
products.filter(p => p.price > 50);
```

Loading, Error and Empty UI States

Why UI States Matter

Real network requests take time, and they can fail. A well-designed interface never leaves the user staring at a blank, frozen or confusing screen while this happens. Every screen that depends on data from a server should be planned around several possible states, not only the ideal “happy path” where everything succeeds immediately.

Loading	Success	Empty	Error
---------	---------	-------	-------

The Loading State

```
async function loadProducts() {
  showLoading(true);
  try {
    const res = await fetch(url);
    const data = await res.json();
    renderProducts(data);
  } finally {
    showLoading(false);
  }
}
```

A spinner or skeleton placeholder should appear the moment a request begins. The finally block always runs, regardless of whether the request ultimately succeeded or failed, making it the ideal place to hide the loading indicator.

The Error State

```
catch (error) {
  showErrorMessage(
    "Could not load products. Please try again."
  );
}
```

Good error messages are clear and written in plain language, and often offer a “Retry” action, rather than displaying a raw technical error directly to the user. As noted earlier, a 404 or 500 response will not automatically be caught unless `response.ok` is checked and an error is deliberately thrown.

The Empty State

```
if (products.length === 0) {
  showEmptyMessage("No products found.");
  return;
}
renderProducts(products);
```

An empty state differs from an error state in one crucial respect: the request itself succeeded, but there is simply no data to display, such as a search returning zero matches, or a brand-new user's empty shopping cart. A friendly message should always be shown here, rather than a blank screen that a user might mistake for a broken page.

The Success State — Rendering Data

```
function renderProducts(products) {  
  const list = document.querySelector("#list");  
  list.innerHTML = "";  
  products.forEach((p) => {  
    const item = document.createElement("li");  
    item.textContent = `${p.name} - Rs.${p.price}`;  
    list.appendChild(item);  
  });  
}
```

The Complete State Flow

Request starts -> **LOADING**

Success + data -> **SUCCESS**

Success + no data -> **EMPTY**

Request fails -> **ERROR**

Every data-driven screen in a professional application should be designed around these four states before any code is written. This is a matter of good frontend thinking and planning, not merely JavaScript syntax.

Introduction to Breaking Problems into Smaller Steps

What is Problem Decomposition?

Problem decomposition means breaking one large, overwhelming problem down into a series of small, manageable steps. It is arguably the single most valuable skill in writing any real program — more valuable, in practice, than memorising syntax. Inexperienced programmers often freeze when faced with a large task; experienced developers immediately begin splitting it into smaller pieces.

Big Problem

Smaller Steps

Tiny, codeable Tasks

Worked Example: Breaking Down a Search Feature

Consider the task: “Build a search box that filters a product list.” It can be broken down as follows:

1. Select the search input and the product list container.
2. Listen for the input event on the search box.
3. On every keystroke, read the currently typed text.
4. Filter the full product array using that text.

5. Re-render only the filtered results to the page.
6. Handle the empty state if nothing matches.

Each of the six steps above is small enough to be written, tested and debugged individually, and together they combine into the full working feature seen earlier in the Interactive UI Behaviour section.

Pseudocode Before Code

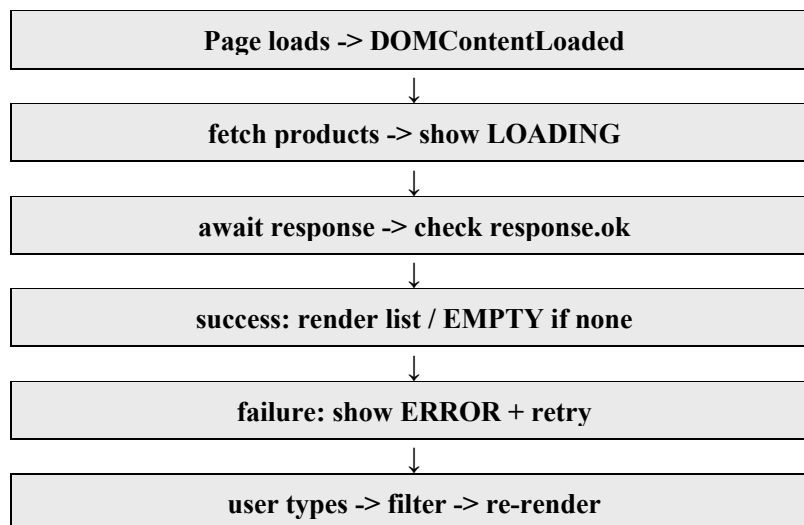
Pseudocode is a plain-English plan for a piece of logic, written before any real code, so that the logic itself can be checked without simultaneously fighting JavaScript syntax.

```
WHEN form is submitted
  IF email empty -> show error
  IF password too short -> show error
  OTHERWISE -> send data to server
  IF server OK -> redirect
  IF server error -> show message
```

Writing pseudocode first makes it far easier to spot missing cases — such as what should happen if the server responds slowly, or fails entirely — and once the plan is solid, it usually translates almost directly into real code, step by step.

Putting It All Together — An Integrated Application

A realistic, data-driven page combines every concept covered in this unit into a single, coherent flow:



The following complete, worked example brings this entire flow together as real code: fetching a list of products, handling every UI state correctly, and allowing the results to be filtered by a live search box.

```
const list = document.querySelector("#productList");
const status = document.querySelector("#status");
const search = document.querySelector("#search");
let allProducts = [];

function renderProducts(products) {
  list.innerHTML = "";
  if (products.length === 0) {
    status.textContent = "No products found.";
    return;
  }
  status.textContent = "";
  products.forEach((p) => {
    const item = document.createElement("li");
    item.textContent = `${p.name} - Rs.${p.price}`;
    list.appendChild(item);
  });
}

async function loadProducts() {
  status.textContent = "Loading...";
  try {
    const response = await fetch("/api/products");
    if (!response.ok) {
      throw new Error("Server error: " + response.status);
    }
    allProducts = await response.json();
    renderProducts(allProducts);
  } catch (error) {
    status.textContent = "Could not load products. Please try again.";
  }
}

search.addEventListener("input", () => {
  const term = search.value.toLowerCase();
  const filtered = allProducts.filter((p) =>
    p.name.toLowerCase().includes(term)
  );
  renderProducts(filtered);
});

document.addEventListener("DOMContentLoaded", loadProducts);
```

Tracing this program from the bottom upward, since that is the order events actually occur in: `DOMContentLoaded` fires once the page's HTML has fully loaded, which calls `loadProducts()`. That function immediately shows a loading message, then awaits the `fetch()` call; if the server responds with an error status such as 404 or 500, `response.ok` is false and an error is deliberately thrown so that the catch block — and therefore the same error-handling code — handles both network failures and HTTP error responses. On success, the parsed JSON array is stored in the outer `allProducts` variable (so that it remains available later for filtering) and passed to `renderProducts()`, which itself checks for the empty state before building and inserting one list item per product. Finally, the search box's input listener filters the already-loaded `allProducts` array on every keystroke and re-renders only the matching subset, without ever needing to contact the server again. This one short program deliberately exercises DOM selection, element creation, the Fetch API, `async/await`, JSON parsing, all four UI states, and the `filter()` array method — the complete set of ideas covered across this unit.

Debugging Asynchronous Code

Asynchronous code can be harder to debug than ordinary synchronous code, because the order in which `console.log()` statements appear may not match the order the corresponding lines appear in the source file. Two habits make this easier to manage. First, logging both the request being made and the response received — for example `console.log("Response status:", response.status)` immediately after a `fetch()` call — makes it obvious whether a problem lies in the request itself or in handling its result. Second, the Network panel inside the browser's developer tools lists every request the page has made, together with its status code, timing, and the exact data sent and received, and is often faster to consult than adding further `console.log()` statements, particularly when trying to determine whether a request even reached the server at all.

Common Mistakes to Avoid — Asynchronous JavaScript

- **Forgetting that `fetch()` does not reject on HTTP errors:** a 404 or 500 response must be detected manually by checking `response.ok`, not assumed to land inside a `.catch()` block or catch clause.
- **Using `await` outside an `async` function:** `await` is only valid inside a function explicitly marked `async`; using it elsewhere is a syntax error.
- **Forgetting to `await response.json()`:** since it also returns a Promise, omitting `await` leaves a pending Promise object instead of the actual parsed data.

- **Not handling the loading state:** a page that gives no visual feedback while data is being fetched can appear broken or unresponsive to the user, even though it is working correctly.
- **Not handling the empty state separately from the error state:** a successful request that legitimately returns zero results should never be presented to the user as if something had gone wrong.
- **Mixing `async/await` with `.then()` unnecessarily:** while technically possible, mixing the two styles in the same function reduces readability; one style should be chosen and used consistently within a given function.

QUICK REFERENCE TABLES

The following tables collect together, in one place, the most frequently used tools covered in this unit, for quick revision before an examination or while working on an assignment.

Quick Reference: Array Methods

Method	Returns	Changes original?
push() / pop()	the new length / the removed item	Yes
map()	a new array of transformed values	No
filter()	a new array of matching values	No
reduce()	a single accumulated value	No
find()	the first matching item, or undefined	No
forEach()	nothing (undefined)	No
some() / every()	a boolean	No
sort()	the same array, reordered	Yes
includes()	a boolean	No

Quick Reference: DOM Methods

Method / Property	Purpose
document.querySelector()	select the first matching element
document.querySelectorAll()	select all matching elements
element.textContent	read or set plain text
element.innerHTML	read or set HTML markup (never with raw user input)
element.classList.add/remove/toggle()	manage CSS classes
document.createElement()	create a new element
parent.appendChild(child)	insert an element into the page
element.remove()	remove an element from the page
element.addEventListener()	attach a function to run on a given event

Quick Reference: Asynchronous JavaScript

Tool	Purpose
Promise	an object representing a value available in the future
.then() / .catch()	run code on success / on failure of a Promise
async function	a function that automatically returns a Promise
await	pause inside an async function until a Promise settles
fetch(url)	request data from a server; returns a Promise
response.json()	parse a Response body as JSON; returns a Promise
response.ok / response.status	check whether an HTTP response succeeded
JSON.parse() / JSON.stringify()	convert JSON text to/from a JavaScript value

ADDITIONAL WORKED EXAMPLES

Sorting and Displaying a Leaderboard

Combining `sort()`, `map()` and DOM rendering together is a common real task: taking a raw array of player scores and displaying them on screen ranked from highest to lowest.

```
const players = [
  { name: "Ravi", score: 82 },
  { name: "Asha", score: 95 },
  { name: "Kiran", score: 68 }
];

const ranked = [...players].sort((a, b) => b.score - a.score);

const board = document.querySelector("#leaderboard");
board.innerHTML = "";
ranked.forEach((player, index) => {
  const row = document.createElement("li");
  row.textContent = `${index + 1}. ${player.name} - ${player.score}`;
  board.appendChild(row);
});
```

The comparison function passed to `sort()`, `(a, b) => b.score - a.score`, arranges the array from the highest score to the lowest: whenever b's score is larger than a's, the result is positive, which tells `sort()` that b should come before a. The spread syntax `[...players]` is used to copy the array before sorting, since `sort()` otherwise rearranges the original array in place — copying first preserves the original, unsorted order of

players in case it is still needed elsewhere in the program. The `forEach()` loop that follows then builds one row per player, using the loop's own index parameter to display each player's rank.

A Simple Accordion / FAQ Panel

An accordion, in which clicking a question reveals or hides its answer, is another very common interactive UI pattern built entirely from `classList` and event delegation.

```
const faq = document.querySelector("#faq");

faq.addEventListener("click", (e) => {
  if (e.target.classList.contains("faq-question")) {
    e.target.nextElementSibling.classList.toggle("open");
  }
});
```

Each question element is expected to be immediately followed in the HTML by its answer element; `nextElementSibling` walks from the clicked question to that following answer, and `toggle("open")` shows or hides it — typically paired with a CSS rule such as `.answer { display: none; }` and `.answer.open { display: block; }`. As with the to-do list example earlier, a single delegated listener on the shared `faq` container automatically continues to work even if further question-and-answer pairs are added to the page later. It is worth noting that HTML5 also provides a native `<details>` and `<summary>` element pair capable of this exact behaviour without any JavaScript at all; the manual version shown here remains useful to know because it generalises to interface patterns, such as tabs, that native HTML elements cannot fully express.

Fixing Common JSON Mistakes

Because JSON's syntax rules are stricter than ordinary JavaScript object syntax, certain small mistakes are extremely common when writing JSON by hand — for example, in a written examination answer, or while manually typing sample data during testing.

```
{ name: "Asha" }           // WRONG – name needs double quotes
{ "name": 'Asha' }         // WRONG – single quotes are not allowed
{ "name": "Asha", }        // WRONG – trailing comma not allowed
{ "name": "Asha" }         // CORRECT
```

All three incorrect examples above would work perfectly well as ordinary JavaScript object literal syntax, which is precisely why the distinction is easy to miss; JSON, however, is a stricter, language-independent text format, and `JSON.parse()` will throw an error on any of the three malformed examples shown.

IMPORTANT QUESTIONS

2-Mark Questions — Short Answer

- 1) What is JavaScript? State whether it is a client-side or server-side language by default.
- 2) Differentiate between `let`, `const` and `var`.
- 3) What is the difference between `undefined` and `null`?
- 4) What is the difference between `==` and `===` in JavaScript?
- 5) What is the DOM? Expand the abbreviation.
- 6) Differentiate between `textContent` and `innerHTML`.
- 7) What is an event in JavaScript? Give two examples of event types.
- 8) What is event bubbling?
- 9) What is the purpose of `preventDefault()`?
- 10) What is form validation? Why is client-side validation not sufficient on its own?
- 11) What is a callback function?
- 12) What is a Promise? Name its three states.
- 13) What is the purpose of the `await` keyword?
- 14) What is the Fetch API used for?
- 15) What is JSON? Give its full form.
- 16) Differentiate between `JSON.parse()` and `JSON.stringify()`.
- 17) What is a loading state in a user interface? Why is it needed?
- 18) What is problem decomposition?

6/8-Mark Questions — Long Answer

- 1) Explain the JavaScript data types with suitable examples for each.
- 2) Explain the control flow statements if/else, switch and the for loop with examples.
- 3) Explain functions in JavaScript. Discuss function declarations, function expressions and arrow functions with examples.
- 4) Explain the array methods map(), filter() and reduce() with suitable examples.
- 5) Explain how to select and update elements in the DOM using querySelector, textContent and classList.
- 6) Explain the event object and event bubbling with a suitable example.
- 7) Explain the process of validating a form using JavaScript, including the use of regular expressions.
- 8) Explain synchronous and asynchronous JavaScript with suitable examples.
- 9) Explain Promises and their three states with an example of .then() and .catch().
- 10) Explain async/await and compare it with the .then()/catch() approach.
- 11) Explain the Fetch API and describe how to handle a JSON response using async/await.
- 12) Explain the loading, error, empty and success states of a data-driven user interface.

8/12-Mark Questions — Detailed Long Answer

- 1) Explain JavaScript fundamentals including variables, data types, operators and control flow with suitable examples.
- 2) Explain functions in JavaScript, covering declarations, expressions, arrow functions, default parameters, rest parameters and spread syntax.
- 3) Explain arrays in JavaScript and discuss the methods map(), filter(), reduce(), find(), some(), every() and sort() with examples.
- 4) Explain objects in JavaScript, including property access, adding/updating/removing properties, methods, the this keyword and destructuring.
- 5) Explain the Document Object Model (DOM) and describe how JavaScript can select and update elements, including a discussion of textContent versus innerHTML and its security implications.
- 6) Explain events in JavaScript, including the event object, event bubbling and capturing, stopPropagation(), preventDefault() and event delegation, with suitable examples.
- 7) Explain form validation in JavaScript, discussing accessing form values, writing custom validation functions, using regular expressions, and providing real-time feedback with the input event.

- 8) Explain asynchronous JavaScript, covering the single-threaded nature of the language, callback functions, the callback hell problem, and how Promises address these limitations.
- 9) Explain Promises in detail, including their states, `.then()/.catch()`, chaining, and `Promise.all()`, with suitable examples.
- 10) Explain `async/await` in JavaScript, including error handling using `try/catch`, and compare it with Promise chaining using suitable examples.
- 11) Explain the Fetch API for API integration, including how `fetch()` returns a Promise, how to read a JSON response, how to check `response.ok` and HTTP status codes, and how to send data using a POST request.
- 12) Explain JSON handling in JavaScript, including JSON syntax rules, `JSON.parse()`, `JSON.stringify()`, and working with arrays of JSON objects returned from a real API.
- 13) Explain the design of loading, error, empty and success states in a user interface, and describe how these states should be implemented when fetching data using `async/await`.
- 14) Explain the concept of problem decomposition and pseudocode, and demonstrate with a worked example how a feature such as a live search box can be broken down into smaller steps and implemented in JavaScript.
- 15) Design a small interactive web application that fetches a list of items from an API, displays loading, error and empty states appropriately, and allows the user to filter the displayed items using a search box. Explain the JavaScript concepts used at each stage.